

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

Дисциплина:
«Алгоритмы и структуры данных»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

Топологическая сортировка по схеме Кана.

Поиск по схеме Кнута-Морриса-Пратта.

Оценка сложности

Выполнил:
Студент гр. N3248
Жук Анастасия Валерьевна

Подпись: _____

Проверил:
Ерофеев Сергей Анатольевич

Подпись: _____

Санкт-Петербург
2025 г.

Техническое задание

Разработать программу топологической сортировки по схеме Кана с последующим поиском по схеме Кнута-Морриса-Пратта, записывая числа в дек на базе связного списка. Оценить сложность.

Переменные

Константы (#define)

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
INITIAL_BUF_SIZE	Начальный размер массива для чтения строк	int	100	4

Структуры данных

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
Node	Узел двусвязного списка	Структура	-	24
edge	Значение ребра (0 или 1)	int	0, 1	4
prev, next	Указатели на соседние узлы	Node*	Адреса узлов или NULL	8
Deque	Дек (двусторонняя очередь)	Структура	-	24
front, rear	Указатели на начало/конец дека	Node*	Адреса узлов или NULL	8
size	Количество элементов в деке	int	от -2 147 483 648 до 2 147 483 647	4

Функции работы с деком

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
---------------------	-------------	-----	----------	---------------

dq	Указатель на дек	Deque*	Адреса или NULL	8
value	Значение для добавления/удаления	int	-1, 0, 1	4
newNode	Указатель на новый узел	Node*	Адреса или NULL	8
temp	Временный указатель на узел	Node*	Адреса или NULL	8

Чтение и обработка матрицы

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
filename	Имя файла с матрицей	char*	"matrix.txt"	Длина строки + 1
file	Указатель на файл	FILE*	Адреса или NULL	8
rows, cols	Размеры матрицы	int	от 0 до 2 147 483 647	4
line	Буфер для чтения строки	char*	Динамическая строка	Длина строки line
ptr, token	Указатели для разбора строки	char*	Адреса в line	8
current_cols	Счетчик столбцов в строке	int	от 0 до 2 147 483 647	4
value	Значение элемента матрицы	double	0.0 или 1.0	8

Топологическая сортировка (алгоритм Кана)

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
matrix	Указатель на дек с матрицей	Deque*	Адреса или NULL	8
n	Размер матрицы	int	от 0 до 2 147 483 647	4
result_size	Размер результата сортировки	int*	от 0 до 2 147 483 647	4

in_degree	Массив полустепеней захода	int*	от 0 до 2 147 483 647	4 * n
zero_in_degree	Дек для вершин степени 0	Deque*	Адреса или NULL	24
result	Результат сортировки	int*	от 0 до 2 147 483 647	4 * n
v, i, j	Временные индексы	int	от 0 до 2 147 483 647	4

Поиск подстроки (алгоритм КМП)

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
text	Текст для поиска	char*	Динамическая строка	Длина строки + 1
pattern	Искомая подстрока	char*	Динамическая строка	Длина строки + 1
n, m	Длины текста и образца	int	от 0 до 2 147 483 647	4
lps	Массив префикс-функции	int*	от 0 до 2 147 483 647	4 * m
len	Длина текущего префикса	int	от 0 до 2 147 483 647	4
i, j	Индексы для текста и образца	int	от 0 до 2 147 483 647	4

Динамические строки и ввод

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
buf_size	Текущий размер буфера	size_t	от 0 до 2 ⁶⁴	8
buffer	Буфер для чтения строки	char*	Динамический массив	buf_size
pos	Текущая позиция в буфере	size_t	от 0 до 2 ⁶⁴	8

c	Считанный символ	int	от 0 до 2 147 483 647	4
---	------------------	-----	-----------------------	---

Подсчет времени работы программы

Название переменной	Расшифровка	Тип	Диапазон	Емкость, байт
start_time	Время начала выполнения программы	clock_t	от -9 223 372 036 854 775 807 до +9 223 372 036 854 775 807	8
end_time	Время окончания выполнения программы	clock_t	от -9 223 372 036 854 775 807 до +9 223 372 036 854 775 807	8
elapsed_time	Затраченное время (мс)	double	от +/- $1.7 * 10^{-308}$ до $1.7 * 10^{308}$	8

Функциональные маршруты

Перед тем как приступить к самой сортировке, необходимо считать из файла матрицу смежности или весов графа. Для топологической сортировки достаточно только матрицы смежности (матрицы, у которой на пересечении i -ой строки и j -ого столбца стоит 1, если существует дуга из вершины i в вершину j , и 0 в противном случае), но также можно рассмотреть матрицу весов, заменяя вес дуги из вершины i в вершину j на 1. Веса никак не влияют на сортировку, поэтому можем от них избавиться и перейти к матрице смежности.

Итак, для того, чтобы перейти к сортировке, нужно проверить следующие свойства матрицы:

1. Матрица состоит из численных значений, не содержит букв и слов;
2. Матрица не пустая;
3. Матрица квадратная, то есть количество столбцов в ней равно количеству строк;

Если в строке присутствуют пустые строки и/или лишние пробелы, то в процессе считывания матрицы из файла в массив необходимо исключить лишние символы.

На первом этапе нужно проверить содержимое файла, чтобы определить, подходит ли оно нам: содержится ли там матрица и удовлетворяет ли она всем вышеописанным условиям.

Если содержимое файла нам подходит, то мы можем записывать значения матрицы в дек, предварительно заменив веса на 1. Матрица будет храниться в виде длинного двусвязного списка, поэтому для дальнейшей работы с ней нужно запомнить ее размерность.

Когда матрица была успешно считана и записана в дек, можно переходить к самой топологической сортировке по схеме Кана.

Топологическая сортировка — это процесс упорядочивания вершин ациклического ориентированного графа согласно частичному порядку, заданному рёбрами орграфа на множестве его вершин. Алгоритм Кана — один из методов такой сортировки, основанный на постепенном удалении вершин с нулевой полустепенью захода.

Алгоритм топологической сортировки по схеме Кана:

1. Вычисляем полустепень захода (количество входящих рёбер) для каждой вершины.
2. Создаём очередь (или дек) для вершин с нулевой полустепенью захода.

3. Извлекаем вершину v из очереди и добавляем её в результат.
4. Уменьшаем полустепени захода для всех соседей v (т.е. вершин, достижимых из v).
 - Если полустепень захода соседа стала нулевой, добавляем его в очередь.
5. Выполняем 3 и 4 пункты до тех пор, пока очередь не станет пустой
6. Если в результате оказались все вершины — граф ациклический, и топологическая сортировка выполнена.
7. Если остались вершины — граф содержит цикл, и сортировка невозможна.

Проведя сортировку, мы получим массив отсортированных вершин, который нужно переделать в текст, по которому и будет осуществляться поиск введенной пользователем подстроки по схеме Кнута-Морриса-Пратта. Текст будет выглядеть следующим образом: вершина + пробел + вершина + пробел + ... + вершина.

Алгоритм КМП

Алгоритм Кнута — Морриса — Пратта (КМП) — это эффективный алгоритм поиска подстроки в строке. Он был разработан Дональдом Кнутом, Воганом Праттом и Джеймсом Моррисом в 1977 году.

Основная идея алгоритма — использовать информацию о предыдущих сравнениях, чтобы избежать лишних проверок символов. Для этого строится префикс-функция (или таблица частичных совпадений), которая помогает определить, на какое максимальное расстояние можно сдвинуть шаблон при несовпадении символов.

Префикс-функция $Prefix(P, i)$ для строки P (шаблона) длины m — это такой массив, в котором каждый его элемент — это длина наибольшего собственного (не равного всей подстроке) суффикса подстроки $P[0..i]$, который одновременно является её префиксом (то есть его бордера).

$$Prefix(P, i) = \max\{k \mid 0 \leq k < i, P[0..k-1] = P[i-k+1..i]\}$$

Алгоритм выглядит следующим образом:

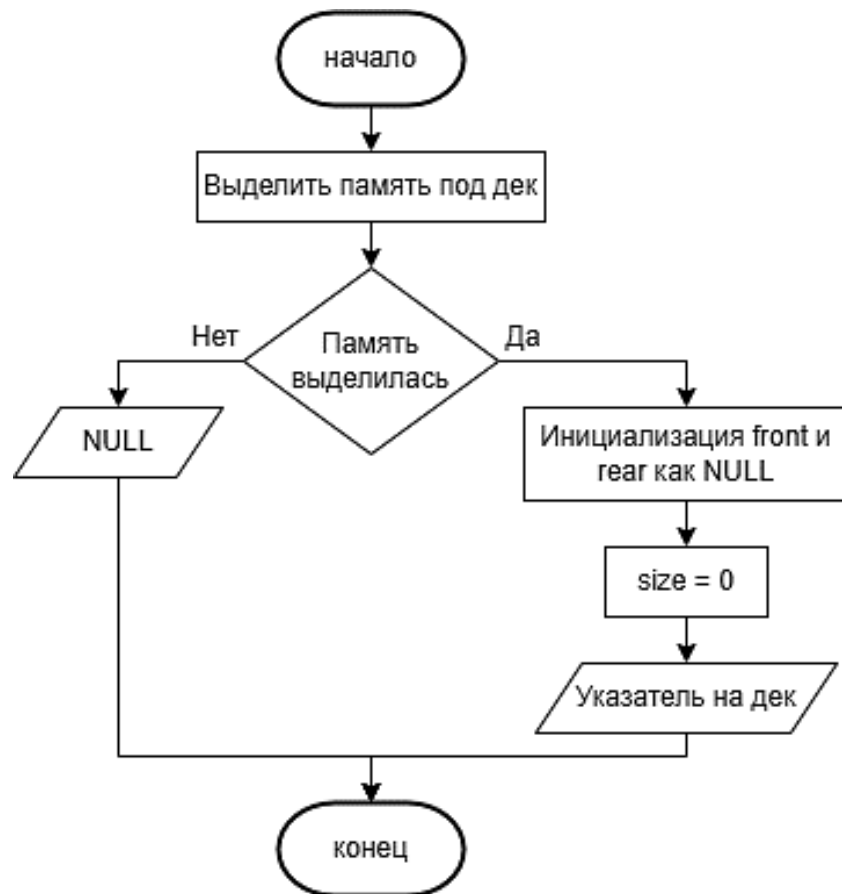
Начальные условия:

- Текст T длины n
 - Шаблон P длины m
1. Вычислить префикс-функцию $Prefix(P, i)$.
 2. Поиск в тексте T :
 - Инициализировать $i = 0$ (индекс в T) и $j = 0$ (индекс в P).
 - Пока $i < n$:
 - Если $T[i] == P[j]$, увеличиваем i и j .

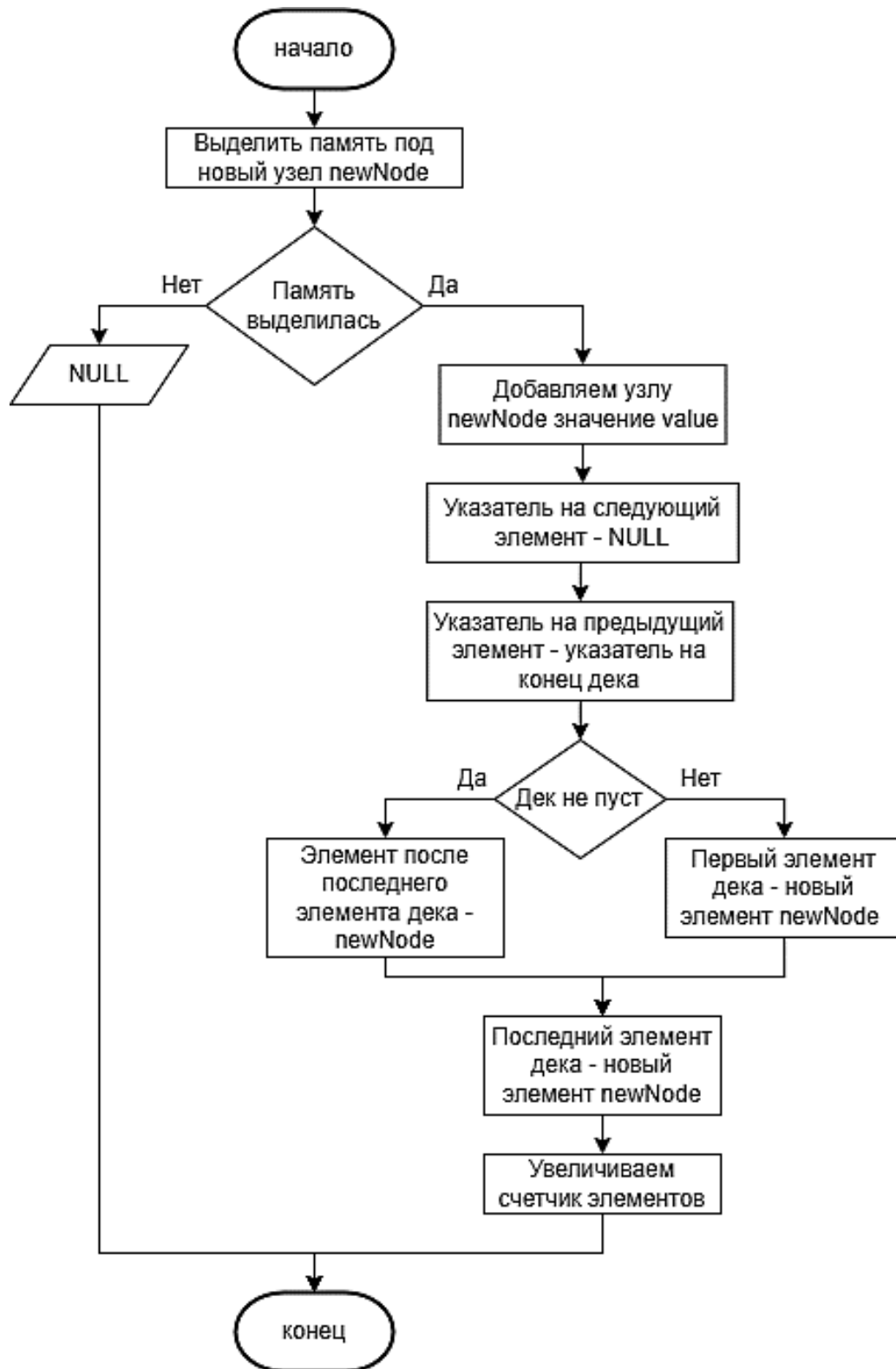
- Если $j == m$, значит, найдено вхождение P в T на позиции $i - j$.
- Если $i < n$ и $T[i] != P[j]$:
 - Если $j > 0$, сдвигаем $j = \text{Prefix}(P, i - 1)$ (используем префикс-функцию).
 - Иначе увеличиваем i .

Блок-схема

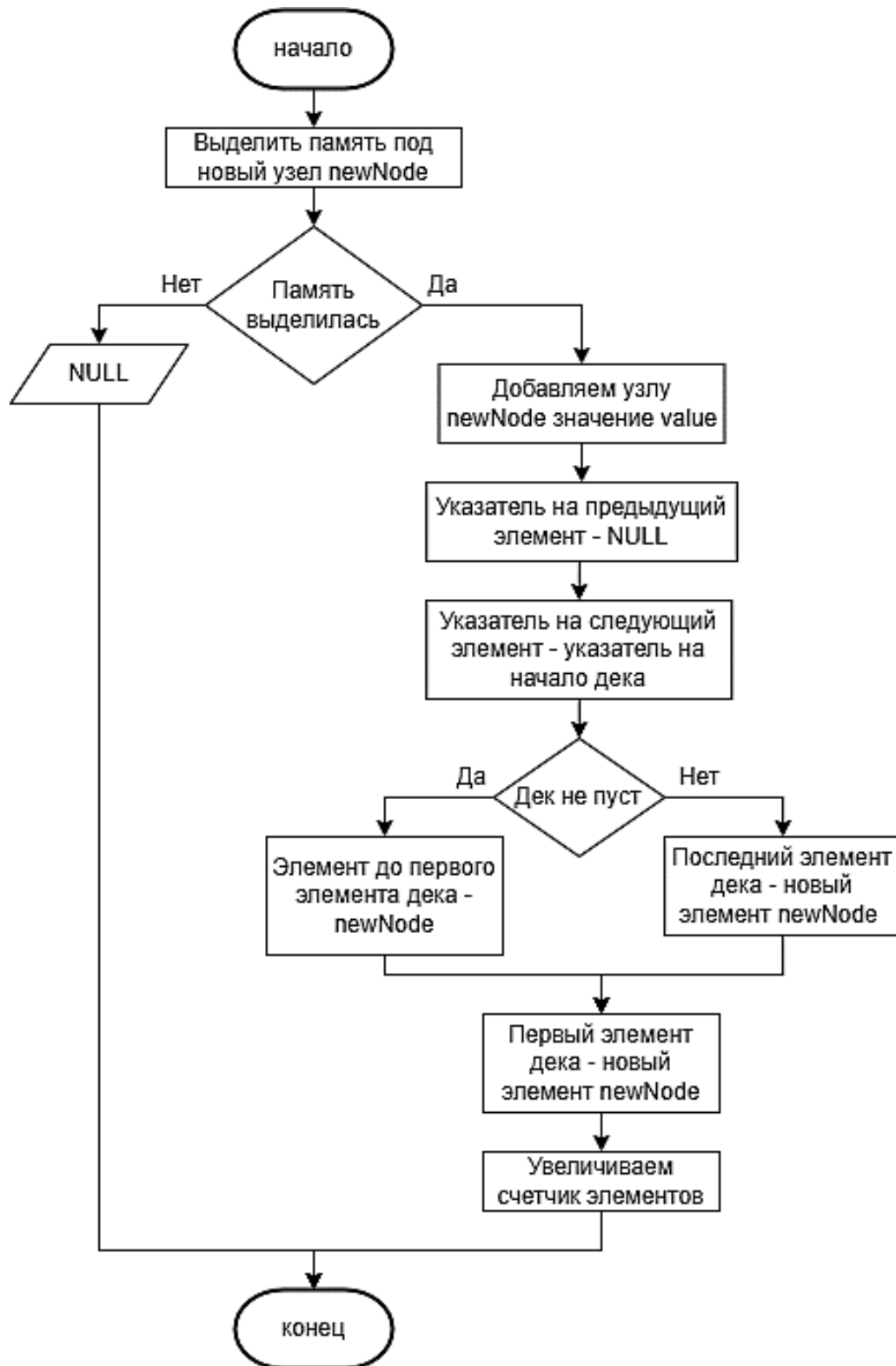
Функция `create_deque()`



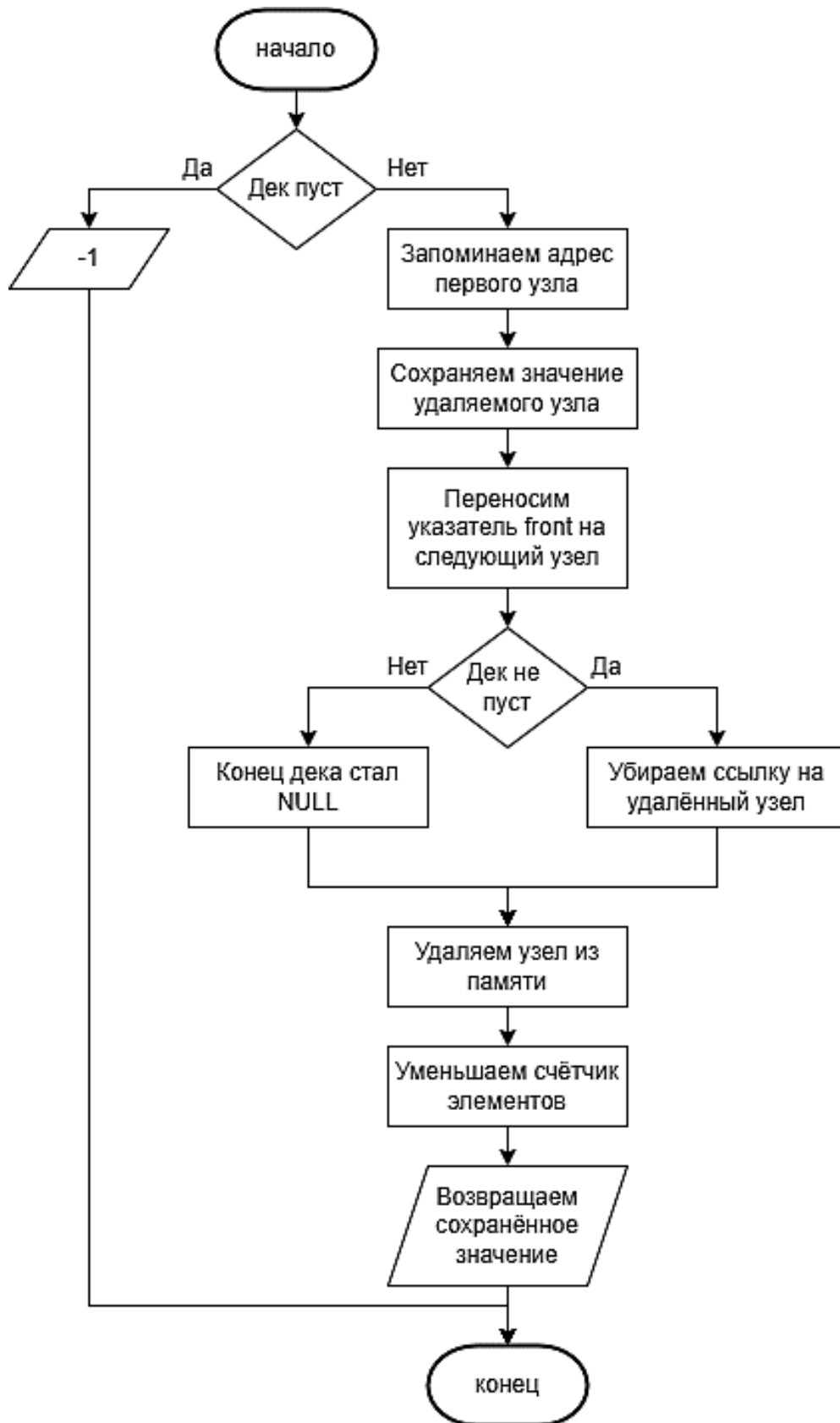
Функция **push_back(Deque* dq, int value)**



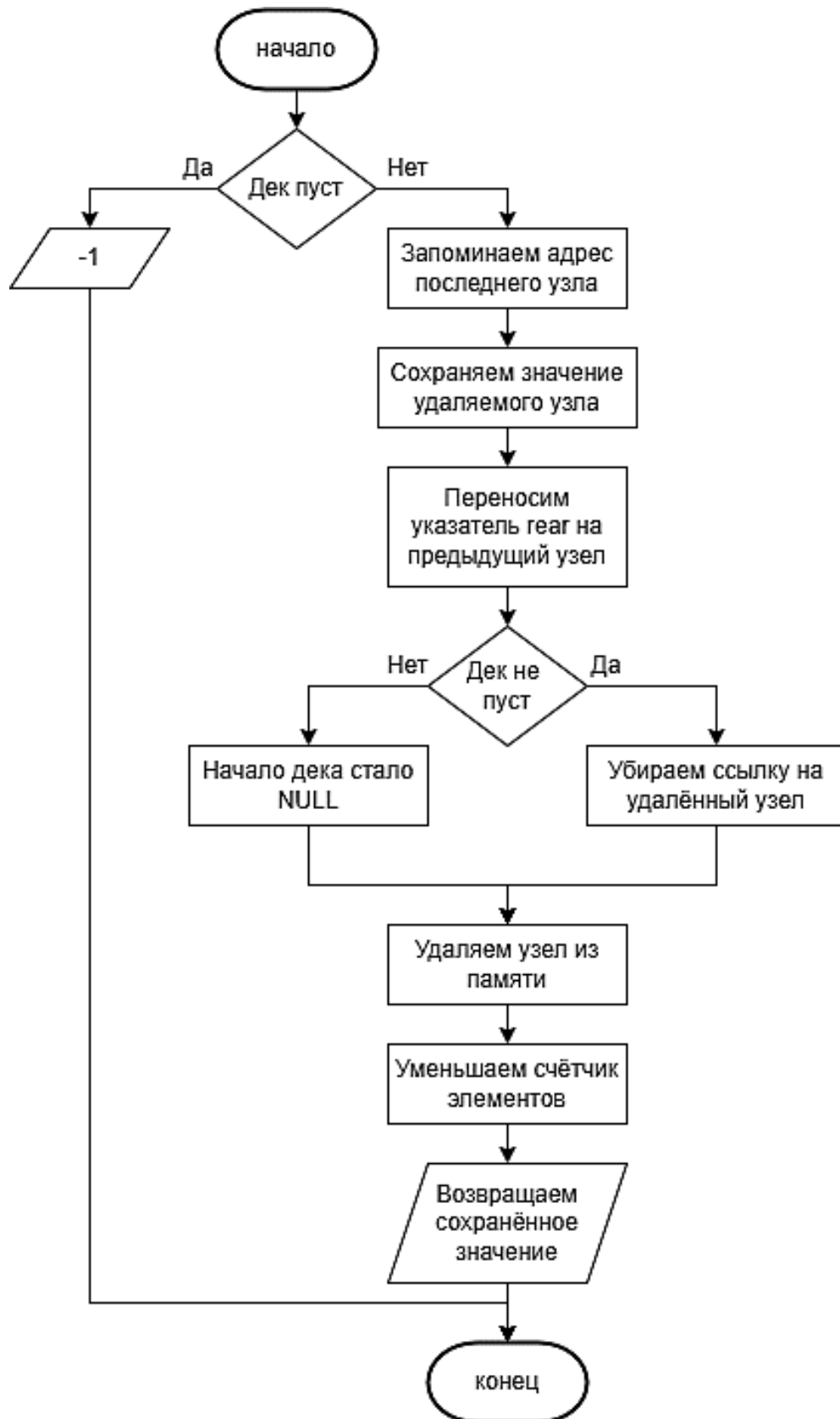
Функция **push_front(Deque* dq, int value)**



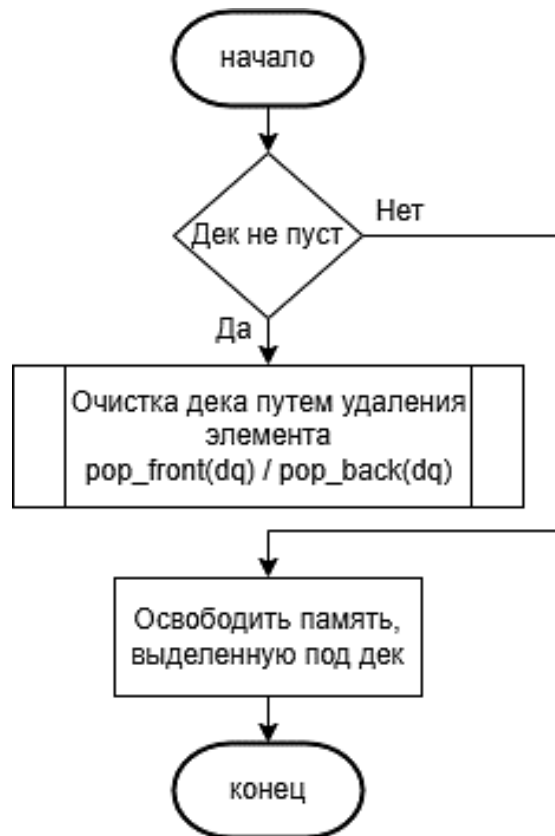
Функция **pop_front(Deque* dq)**



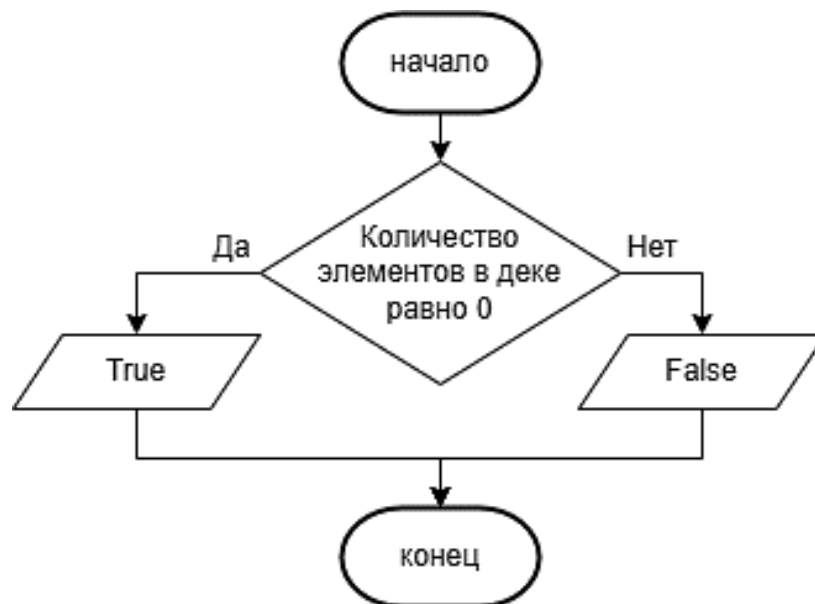
Функция `pop_back(Deque* dq)`



Функция **free_deque(Deque* dq)**



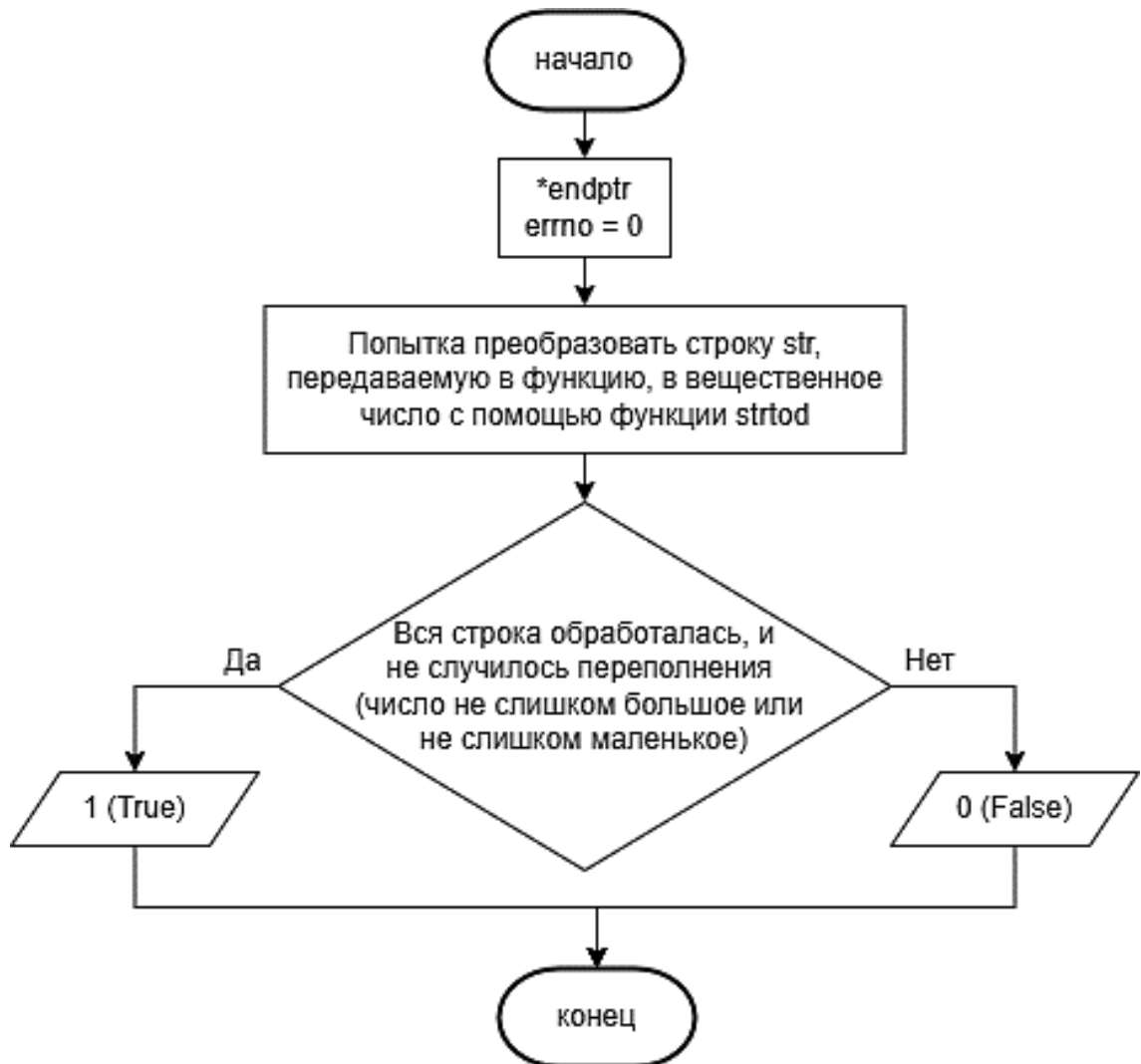
Функция **is_empty(const Deque* dq)**



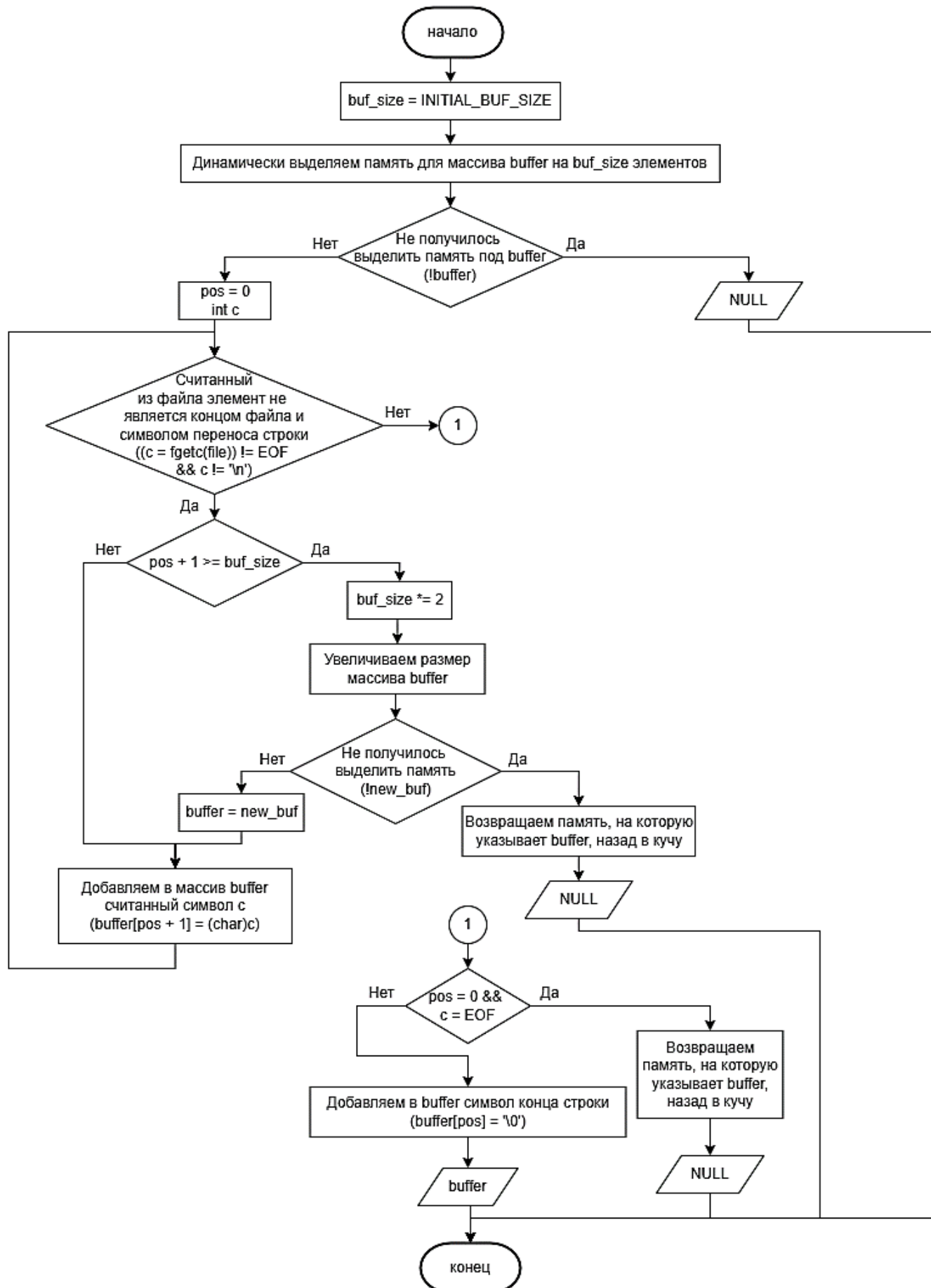
Функция `get_element(Deque* dq, int row, int col, int n)`



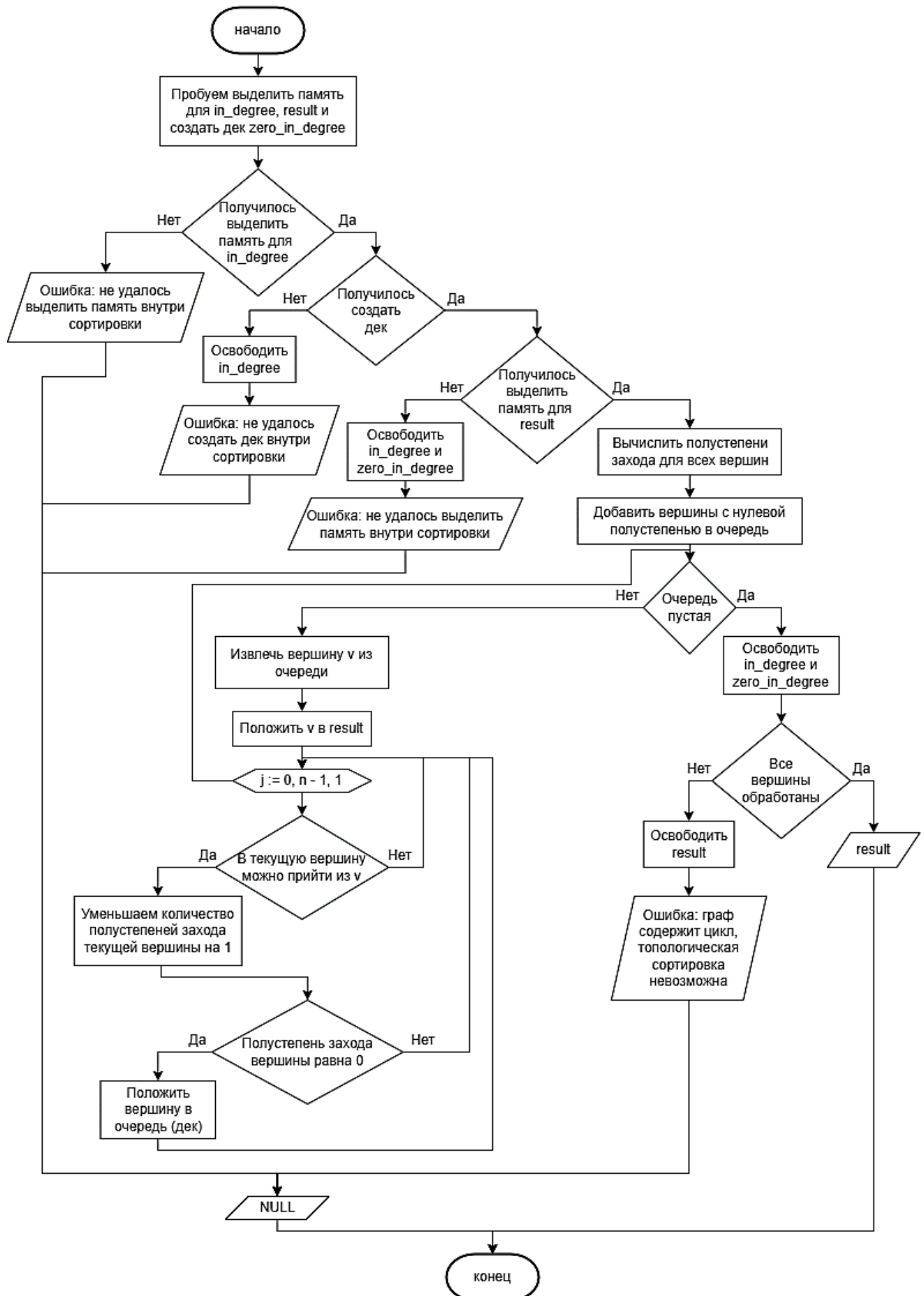
Функция `is_valid_number(const char *str)`



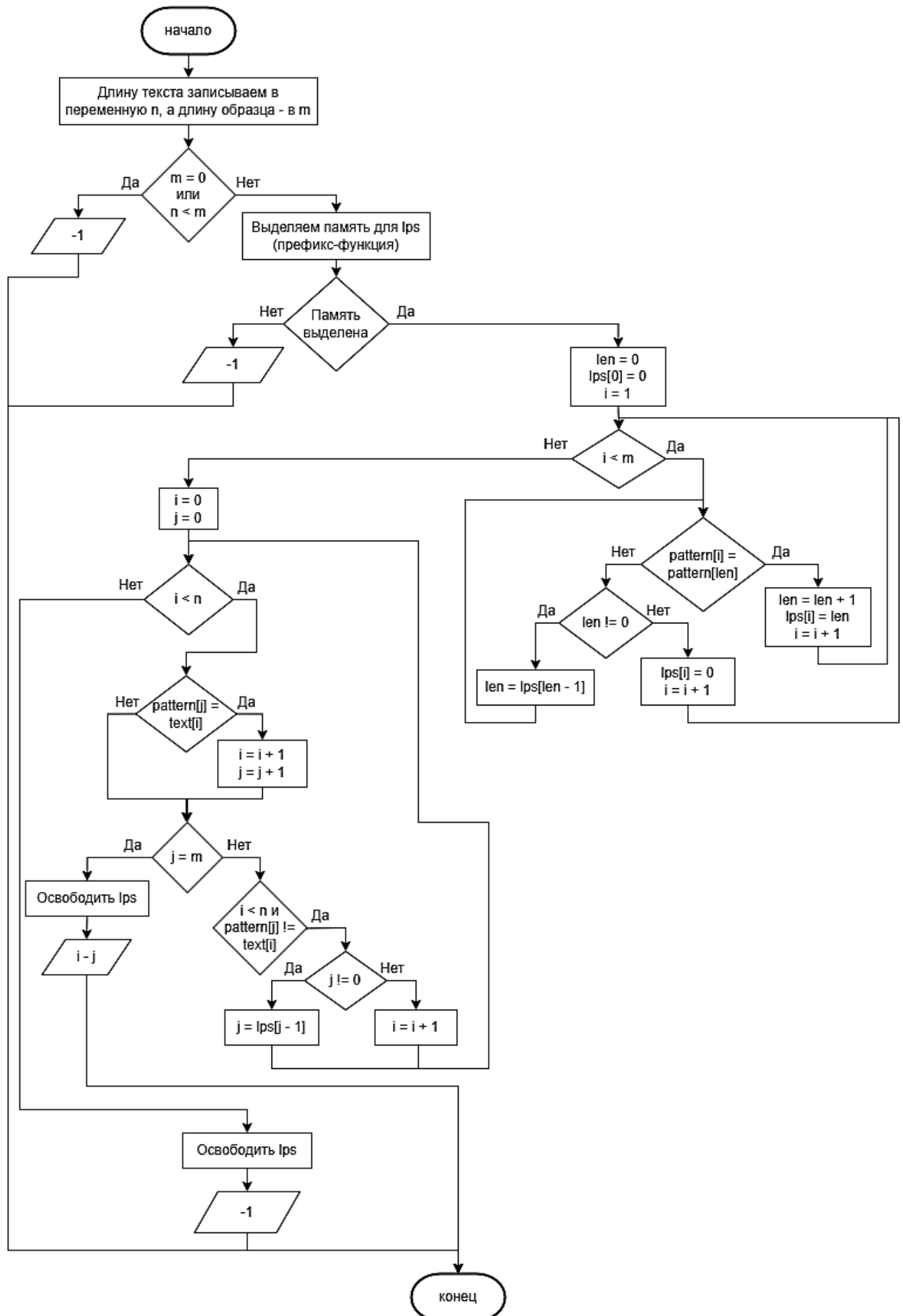
Функция `read_dynamic_line(FILE* file)`



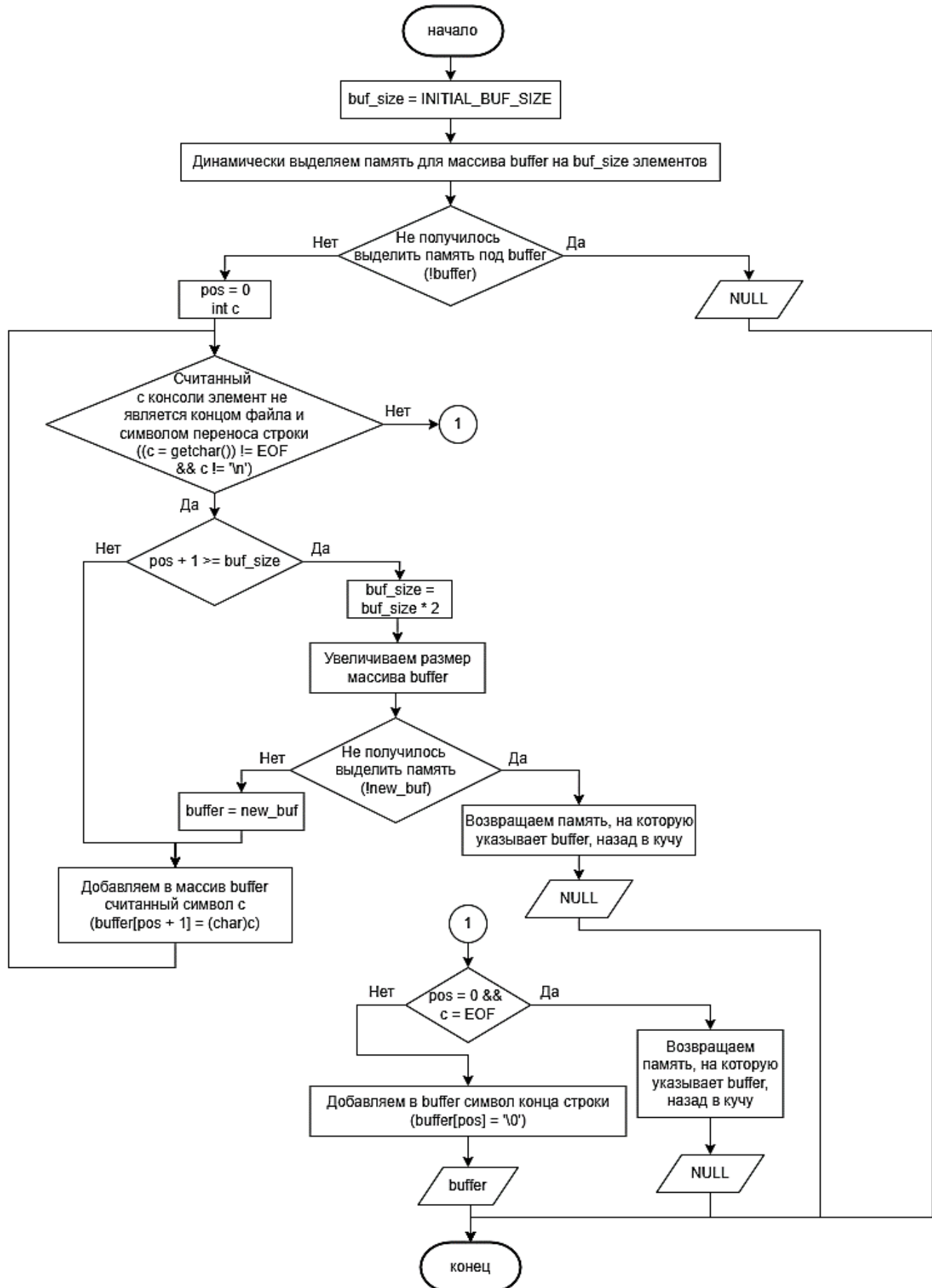
Функция **kahn_topological_sort(Deque* matrix, int n, int* result_size)**



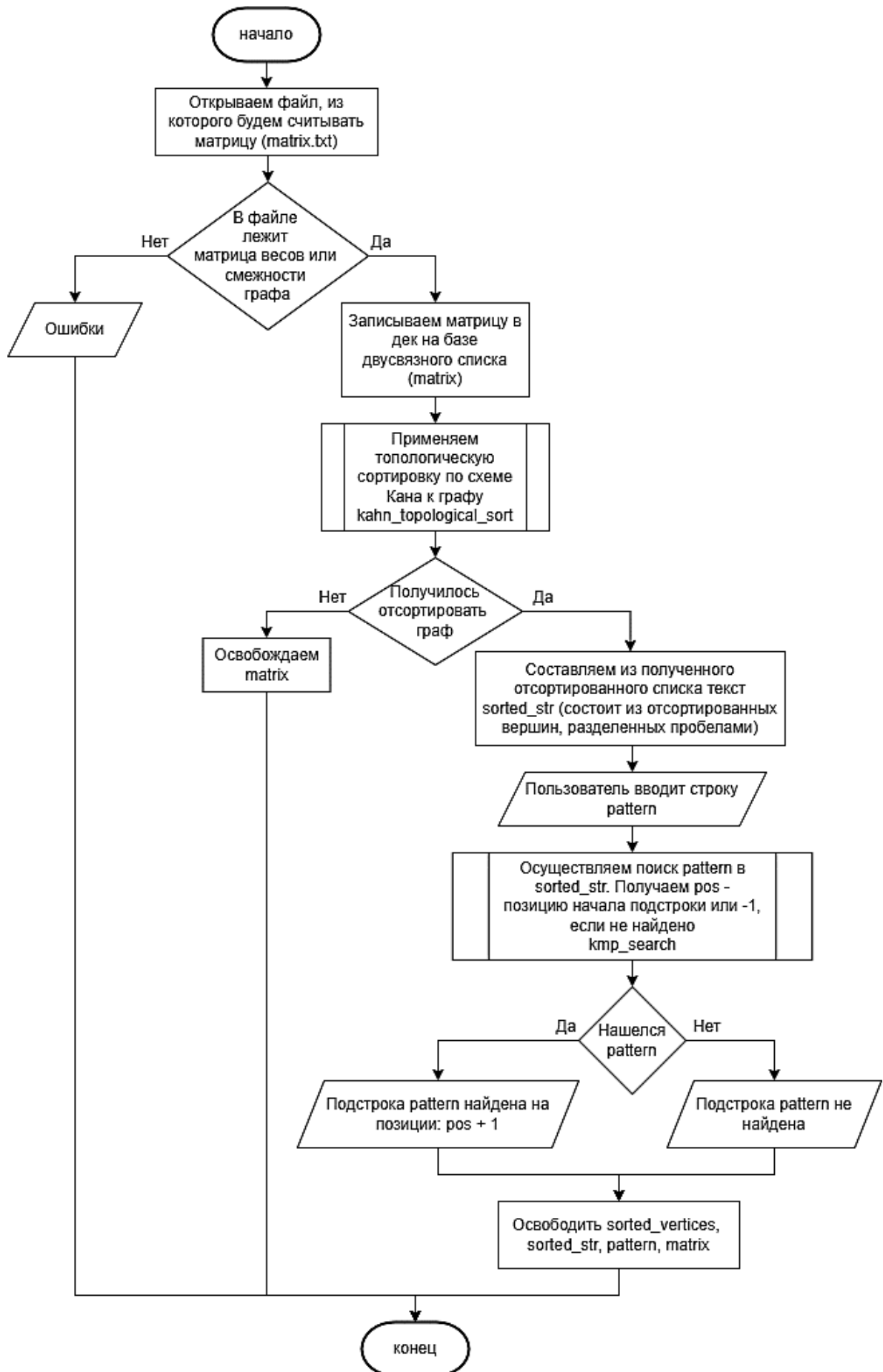
Функция `kmp_search(const char* text, const char* pattern)`



Функция read_input_line()



Основная функция



Код на языке программирования С

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>
#include <time.h>

// Начальный размер буфера для чтения строк
#define INITIAL_BUF_SIZE 100

// Структура узла двусвязного списка
typedef struct Node {
    int edge; // Значение ребра (0 или 1)
    struct Node* prev; // Указатель на предыдущий узел
    struct Node* next; // Указатель на следующий узел
} Node;

// Структура дека (двусторонней очереди)
typedef struct Deque {
    Node* first; // Указатель на начало дека
    Node* last; // Указатель на конец дека
    int size; // Текущее количество элементов
} Deque;

/*
Создает новый пустой дек
Возвращает указатель на созданный дек или NULL при ошибке выделения памяти
*/
Deque* create_deque() {
    Deque* dq = malloc(sizeof(Deque));
    if (!dq) return NULL; // Проверка успешности выделения памяти
    dq->first = dq->last = NULL; // Инициализация пустого дека
    dq->size = 0;
    return dq;
}

/*
Добавляет элемент в конец дека
dq - указатель на дек
value - значение для добавления
*/
void push_back(Deque* dq, int value) {
    Node* newNode = malloc(sizeof(Node));
    if (!newNode) return; // Проверка выделения памяти
```

```

// Инициализация нового узла
newNode->edge = value;
newNode->next = NULL;
newNode->prev = dq->last;

// Обновление связей в деке
if (dq->last) {
    dq->last->next = newNode; // Если дек не пуст
} else {
    dq->first = newNode; // Если дек пуст
}
dq->last = newNode; // Новый элемент теперь последний
dq->size++; // Увеличиваем счетчик элементов
}

/*
Добавляет элемент в начало дека
dq - указатель на дек
value - значение для добавления
*/
void push_front(Deque* dq, int value) {
    Node* newNode = malloc(sizeof(Node));
    if (!newNode) return; // Проверка выделения памяти

    // Инициализация нового узла
    newNode->edge = value;
    newNode->prev = NULL;
    newNode->next = dq->first;

    if (dq->first) {
        dq->first->prev = newNode; // Если дек не пуст
    } else {
        dq->last = newNode; // Если дек пуст
    }
    dq->first = newNode;
    dq->size++;
}

/*
Удаляет элемент из начала дека
dq - указатель на дек
Возвращает значение удаленного элемента или -1, если дек пуст
*/
int pop_front(Deque* dq) {
    if (!dq->first) return -1; // Проверка на пустоту дека

    Node* temp = dq->first; // Сохраняем ссылку на удаляемый узел
    int value = temp->edge; // Сохраняем значение

```

```

dq->first = dq->first->next; // Перемещаем указатель на следующий элемент

// Обновляем связи
if (dq->first) {
    dq->first->prev = NULL; // Удаляем ссылку на предыдущий элемент
} else {
    dq->last = NULL; // Если дек стал пустым
}

free(temp); // Освобождаем память
dq->size--; // Уменьшаем счетчик элементов
return value; // Возвращаем значение
}

/*
Удаляет элемент из конца дека
dq - указатель на дек
Возвращает значение удаленного элемента или -1, если дек пуст
*/
int pop_back(Deque* dq) {
    if (!dq->first) return -1; // Проверка на пустоту дека

    Node* temp = dq->last; // Сохраняем ссылку на удаляемый узел
    int value = temp->edge; // Сохраняем значение
    dq->last = dq->last->prev; // Перемещаем указатель на следующий элемент

    // Обновляем связи
    if (dq->last) {
        dq->last->next = NULL; // Удаляем ссылку на предыдущий элемент
    } else {
        dq->first = NULL; // Если дек стал пустым
    }

    free(temp); // Освобождаем память
    dq->size--; // Уменьшаем счетчик элементов
    return value; // Возвращаем значение
}

/*
Освобождает память, занятую деком
dq - указатель на дек
*/
void free_deque(Deque* dq) {
    while (dq->first) { // Пока дек не пуст
        pop_front(dq); // Удаляем элементы из начала
    }
    free(dq); // Освобождаем память структуры дека
}

```



```

/*
Проверяет дек на пустоту
dq - указатель на дек
Возвращает true, если дек пуст, иначе false
*/
bool is_empty(const Deque* dq) {
    return dq->size == 0;
}

/*
Получает элемент матрицы по индексам строки и столбца
dq - указатель на дек с матрицей
row - индекс строки
col - индекс столбца
n - размер матрицы (n x n)
Возвращает значение элемента матрицы или 0 при ошибке
*/
int get_element(Deque* dq, int row, int col, int n) {
    Node* current = dq->first;
    int pos = row * n + col; // Линейная позиция в deque

    // Переход к нужной позиции
    for (int i = 0; i < pos && current; i++) {
        current = current->next;
    }
    return current ? current->edge : 0; // Возвращаем значение или 0 если NULL
}

/*
Проверяет, является ли строка корректным числом
str - проверяемая строка
Возвращает 1, если строка - корректное число, иначе 0
*/
int is_valid_number(const char *str) {
    char *endptr;
    errno = 0;
    strtod(str, &endptr); // Пытаемся преобразовать строку в число

    // Проверяем два условия:
    // 1. Преобразование дошло до конца строки или пробела
    // 2. Не было переполнения
    return (*endptr == '\0' || isspace(*endptr)) && errno != ERANGE;
}

// Считываем строку из файла

/*

```

Читает строку произвольной длины из файла

file - указатель на файл

Возвращает указатель на прочитанную строку или NULL при ошибке

*/

```
char* read_dynamic_line(FILE* file) {
    size_t buf_size = INITIAL_BUF_SIZE;
    char* buffer = malloc(buf_size);
    if (!buffer) return NULL; // Проверка выделения памяти
```

```
    size_t pos = 0;
```

```
    int c;
```

```
    // Читаем символы до конца строки или файла
```

```
    while ((c = fgetc(file)) != EOF && c != '\n') {
```

```
        // Если буфер заполнен, увеличиваем его размер
```

```
        if (pos + 1 >= buf_size) {
```

```
            buf_size *= 2;
```

```
            char* new_buf = realloc(buffer, buf_size);
```

```
            if (!new_buf) {
```

```
                free(buffer);
```

```
                return NULL;
```

```
            }
```

```
            buffer = new_buf;
```

```
        }
```

```
        buffer[pos++] = (char)c; // Записываем символ в буфер
```

```
    }
```

```
    // Если ничего не прочитали и достигли конца файла
```

```
    if (pos == 0 && c == EOF) {
```

```
        free(buffer);
```

```
        return NULL;
```

```
    }
```

```
    buffer[pos] = '\0'; // Добавляем завершающий нулевой символ
```

```
    return buffer;
```

```
}
```

```
// Алгоритм топологической сортировки по схеме Кана
```

```
/*
```

Выполняет топологическую сортировку графа алгоритмом Кана

adj_matrix - матрица смежности графа в виде дека

n - размер матрицы (количество вершин)

result_size - указатель для сохранения размера результата

Возвращает массив с отсортированными вершинами или NULL при наличии цикла

```
*/
```

```
int* kahn_topological_sort(Deque* matrix, int n, int* result_size) {
```

```
    // Выделяем память для полустепеней захода
```

```

int* in_degree = calloc(n, sizeof(int));
if (!in_degree) {
    printf("Ошибка: не удалось выделить память внутри сортировки\n");
    return NULL; // Не удалось выделить память
}

// Создаём очередь вершин с нулевой полустепенью
Deque* zero_in_degree = create_deque();
if (!zero_in_degree) {
    free(in_degree);
    printf("Ошибка: не удалось создать дек внутри сортировки\n");
    return NULL; // Не удалось создать дек
}

// Выделяем память для результата
int* result = malloc(n * sizeof(int));
if (!result) {
    free(in_degree);
    free_deque(zero_in_degree);
    printf("Ошибка: не удалось выделить память внутри сортировки\n");
    return NULL; // Не удалось выделить память
}

*result_size = 0; // Изначально результат пуст

// Вычисляем полустепени захода для всех вершин
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (get_element(matrix, i, j, n)) {
            in_degree[j]++;
        }
    }
}

// Добавляем в очередь вершины с нулевой полустепенью
for (int i = 0; i < n; i++) {
    if (in_degree[i] == 0) {
        push_back(zero_in_degree, i);
    }
}

// Основной цикл алгоритма
while (!is_empty(zero_in_degree)) {
    int v = pop_front(zero_in_degree);
    result[(*result_size)++] = v;

    // Уменьшаем полустепени захода для всех соседей
    for (int j = 0; j < n; j++) {

```

```

        if (get_element(matrix, v, j, n)) {
            in_degree[j]--;
            if (in_degree[j] == 0) {
                push_back(zero_in_degree, j);
            }
        }
    }
}

// Освобождаем память
free(in_degree);
free_deque(zero_in_degree);

// Проверяем, есть ли циклы в графе
if (*result_size != n) {
    free(result);
    printf("Ошибка: граф содержит цикл, топологическая сортировка невозможна\n");
    return NULL; // Граф содержит цикл
}

return result;
}

// Алгоритм Кнута-Морриса-Пратта

/*
Выполняет поиск подстроки в тексте алгоритмом КМП
text - текст для поиска
pattern - искомая подстрока
Возвращает позицию начала подстроки или -1, если не найдено
*/
int kmp_search(const char* text, const char* pattern) {
    int n = strlen(text); // Длина текста
    int m = strlen(pattern); // Длина образца
    if (m == 0 || n < m) return -1; // Проверка на пустые строки

    // Создаем массив для префикс-функции
    int* lps = malloc(m * sizeof(int));
    if (!lps) return -1; // Проверка выделения памяти

    // Вычисляем префикс-функцию для образца
    int len = 0; // Длина текущего префикса
    lps[0] = 0; // Префикс-функция для первого символа всегда 0

    for (int i = 1; i < m; ) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
        }
    }
}

```

```

        i++;
    } else {
        if (len != 0) {
            len = lps[len - 1];
        } else {
            lps[i] = 0;
            i++;
        }
    }
}

// Поиск образца в тексте
int i = 0, j = 0; // Индексы для текста и образца
while (i < n) {
    if (pattern[j] == text[i]) { // Совпадение символов
        i++;
        j++;
    }
    // Если нашли полное совпадение
    if (j == m) {
        free(lps);
        return i - j; // Возвращаем позицию начала совпадения
    }
    // Если символы не совпали
    else if (i < n && pattern[j] != text[i]) {
        if (j != 0) {
            j = lps[j - 1]; // Используем префикс-функцию для сдвига
        } else {
            i++; // Просто двигаемся по тексту
        }
    }
}

free(lps);
return -1; // Совпадение не найдено
}

// Чтение пользовательского ввода

/*
Читает строку с пробелами из стандартного ввода
Возвращает указатель на прочитанную строку или NULL при ошибке
*/
char* read_input_line() {
    size_t buf_size = INITIAL_BUF_SIZE;
    char* buffer = malloc(buf_size);
    if (!buffer) return NULL; // Проверка выделения памяти

```

```

size_t pos = 0;
int c;

// Читаем символы до конца строки или файла
while ((c = getchar()) != EOF && c != '\n') {
    // Если буфер заполнен, увеличиваем его размер
    if (pos + 1 >= buf_size) {
        buf_size *= 2;
        char* new_buf = realloc(buffer, buf_size);
        if (!new_buf) {
            free(buffer);
            return NULL;
        }
        buffer = new_buf;
    }
    buffer[pos++] = (char)c; // Записываем символ в буфер
}

// Если ничего не прочитали и достигли конца файла
if (pos == 0 && c == EOF) {
    free(buffer);
    return NULL;
}

buffer[pos] = '\0'; // Добавляем завершающий нулевой символ
return buffer;
}

int main() {
    // Начинаем считать время
    clock_t start_time = clock();

    const char* filename = "matrix.txt"; // Имя файла с матрицей смежности
    FILE* file = fopen(filename, "r"); // Открываем файл для чтения
    if (!file) {
        perror("Ошибка: не удалось открыть файл");
        return 1;
    }

    // Чтение и проверка матрицы

    int rows = 0, cols = 0; // Счетчики строк и столбцов
    char* line = NULL; // Буфер для чтения строк

    // Первый проход: определяем размеры матрицы
    while ((line = read_dynamic_line(file)) != NULL) {
        if (strlen(line) == 0) { // Пропускаем пустые строки
            free(line);

```

```

        continue;
    }

    // Разбираем строку на числа
    int current_cols = 0;
    char* ptr = line;
    char* token;

    while (*ptr) {
        // Пропускаем пробелы
        while (*ptr && isspace(*ptr)) ptr++;
        if (!*ptr) break; // Конец строки

        // Выделяем токен (число)
        token = ptr;
        while (*ptr && !isspace(*ptr)) ptr++;

        // Проверяем, является ли токен числом
        if (!is_valid_number(token)) {
            printf("Ошибка: некорректное значение '%s'\n", (int)(ptr - token), token);
            free(line);
            fclose(file);
            return 1;
        }
        current_cols++;
    }

    // Проверяем согласованность количества столбцов
    if (cols == 0) {
        cols = current_cols; // Первая строка задает количество столбцов
    } else if (current_cols != cols) {
        printf("Ошибка: разное количество элементов в строках\n");
        free(line);
        fclose(file);
        return 1;
    }
    rows++; // Считаем строки
    free(line);
}

// Проверяем, что матрица не пустая
if (rows == 0 || cols == 0) {
    printf("Ошибка: пустая матрица\n");
    fclose(file);
    return 1;
}

// Проверяем, что матрица квадратная

```

```

if (rows != cols) {
    printf("Ошибка: матрица смежности должна быть квадратной\n");
    fclose(file);
    return 1;
}

// Возвращаемся в начало файла для второго прохода
rewind(file);

// Создание и заполнение дека

// Создаем единый дек для хранения матрицы смежности
Deque* matrix = create_deque();

// Чтение матрицы и сохранение в дек
while ((line = read_dynamic_line(file)) != NULL) {
    if (strlen(line) == 0) {
        free(line);
        continue;
    }

    char* ptr = line;
    char* token;

    while (*ptr) {
        while (*ptr && isspace(*ptr)) ptr++;
        if (!*ptr) break;

        token = ptr;
        while (*ptr && !isspace(*ptr)) ptr++;

        // Преобразуем строку в число и добавляем в дек
        double value = strtod(token, NULL);
        if (value != 0) push_back(matrix, 1); // Ребро есть
        else push_back(matrix, 0); // Ребра нет
    }
    free(line);
}
fclose(file);

// Выводим содержимое дека для демонстрации
printf("Содержимое дека (матрица смежности графа):\n");
Node* current = matrix->first;
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (current) {
            printf("%d ", current->edge);
            current = current->next;
        }
    }
}

```



```

    }
}
printf("\n");
}

// Топологическая сортировка

int result_size;
int* sorted_vertices = kahn_topological_sort(matrix, rows, &result_size);

// Проверяем, удалась ли сортировка
if (!sorted_vertices) {
    free_deque(matrix);
    return 1;
}

// Преобразуем результат сортировки в строку
char* sorted_str = malloc(rows * 12); // Выделяем память с запасом
if (!sorted_str) {
    printf("Ошибка: не удалось выделить память\n");
    free(sorted_vertices);
    free_deque(matrix);
    return 1;
}
sorted_str[0] = '\0'; // Начинаем с пустой строки

// Формируем строку из отсортированных вершин
for (int i = 0; i < result_size; i++) {
    char num_str[12];
    sprintf(num_str, "%d ", sorted_vertices[i] + 1); // +1 для нумерации вершин с 1
    strcat(sorted_str, num_str); // Добавляем к результату
}

// Поиск подстроки

printf("Отсортированные вершины: %s\n", sorted_str);
printf("Введите подстроку для поиска: ");

// Чтение ввода с пробелами
char* pattern = read_input_line();
if (!pattern) {
    printf("Ошибка: не удалось прочитать ввод\n");
    free(sorted_vertices);
    free(sorted_str);
    free_deque(matrix);
    return 1;
}

```

```

// Выполняем поиск и выводим результат
int pos = kmp_search(sorted_str, pattern);
if (pos != -1) {
    printf("Подстрока '%s' найдена на позиции: %d\n", pattern, pos + 1); // +1 для читаемого
формата
} else {
    printf("Подстрока '%s' не найдена\n", pattern);
}

// Освобождение памяти
free(sorted_vertices);
free(sorted_str);
free(pattern);
free_deque(matrix);

// Заканчиваем считать время
clock_t end_time = clock();
double elapsed_time = (double)(end_time - start_time) / CLOCKS_PER_SEC * 1000;
printf("Время выполнения программы: %.3f миллисекунд\n", elapsed_time);

return 0;
}

```

Примеры работы программы

C: > Users > Анастасия > matrix.txt

```
1 0 1 1 1 1
2 0 0 0 1 0
3 0 0 0 1 1
4 0 0 0 0 1
5 0 0 0 0 0
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Анастасия> gcc lab_2_algorithms.c -o program

PS C:\Users\Анастасия> ./program

Содержимое дека (матрица смежности графа):

```
0 1 1 1 1
0 0 0 1 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0
```

Отсортированные вершины: 1 2 3 4 5

Введите подстроку для поиска: 2 3

Подстрока '2 3' найдена на позиции: 3

PS C:\Users\Анастасия>

C: > Users > Анастасия > matrix.txt

```
1
2 | 0 10 1 1 0.1
3
4 0 0 0 -5 0
5 0 0 0 1236 1
6 0 0 0 0 1.0265
7 0 0 0 0 0
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Анастасия> ./program

Содержимое дека (матрица смежности графа):

```
0 1 1 1 1
0 0 0 1 0
0 0 0 1 1
0 0 0 0 1
0 0 0 0 0
```

Отсортированные вершины: 1 2 3 4 5

Введите подстроку для поиска: 3 4 6

Подстрока '3 4 6' не найдена

PS C:\Users\Анастасия>

Оценка сложности

Код с комментариями, в которых подсчитывается количество операций:

```
#include <stdio.h>
#include <stdbool.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <errno.h>

#define INITIAL_BUF_SIZE 100

typedef struct Node {
    int edge;
    struct Node* prev;
    struct Node* next;
} Node;

typedef struct Deque {
    Node* first;
    Node* last;
    int size;
} Deque;

Deque* create_deque() {
    Deque* dq = malloc(sizeof(Deque)); // sizeof + malloc + присваивание = 3
    if (!dq) return NULL;             // проверка условия + return = 2
    dq->first = dq->last = NULL;      // 2 обращения + 2 присваивания = 4
    dq->size = 0;                      // обращение + присваивание = 2
    return dq;                        // return = 1
}                                     // всего (всегда в худшем случае): 3 + 1 (только проверили
// условие) + 4 + 2 + 1 = 11

void push_back(Deque* dq, int value) {
    Node* newNode = malloc(sizeof(Node)); // sizeof + malloc + присваивание = 3
    if (!newNode) return;                 // проверка условия + return = 2

    newNode->edge = value;                 // обращение + присваивание = 2
    newNode->next = NULL;                  // обращение + присваивание = 2
    newNode->prev = dq->last;               // 2 обращения + присваивание = 3

    if (dq->last) {                        // обращение + проверка условия = 2
        dq->last->next = newNode;          // 2 обращения + присваивание = 3
    } else {
        dq->first = newNode;               // обращение + присваивание = 2
    }
    dq->last = newNode;                    // обращение + присваивание = 2
    dq->size++;                             // обращение + инкремент (сложение + присваивание) = 3
}
```

```

} // всего: 3 + 1 (только проверили условие) + 2 + 2 + 3 + 2 + 3
(худшая из ветвей if) + 2 + 3 = 21

```

```

void push_front(Deque* dq, int value) {
    Node* newNode = malloc(sizeof(Node)); // malloc + sizeof + присваивание = 3
    if (!newNode) return; // проверка условия + return = 2

    newNode->edge = value; // обращение + присваивание = 2
    newNode->prev = NULL; // обращение + присваивание = 2
    newNode->next = dq->first; // 2 обращения + присваивание = 3

    if (dq->first) { // обращение + проверка условия = 2
        dq->first->prev = newNode; // 2 обращения + присваивание = 3
    } else {
        dq->last = newNode; // обращение + присваивание = 2
    }
    dq->first = newNode; // обращение + присваивание = 2
    dq->size++; // обращение + инкремент = 3
} // всего: 3 + 1 (только условие) + 2 + 2 + 3 + 2 + 3 (худшая из
ветвей if) + 2 + 3 = 21

```

```

int pop_front(Deque* dq) {
    if (!dq->first) return -1; // обращение + проверка условия + return = 3

    Node* temp = dq->first; // обращение + присваивание = 2
    int value = temp->edge; // обращение + присваивание = 2
    dq->first = dq->first->next; // 3 обращения + присваивание = 4

    if (dq->first) { // обращение + проверка условия = 2
        dq->first->prev = NULL; // 2 обращения + присваивание = 3
    } else {
        dq->last = NULL; // обращение + присваивание = 2
    }

    free(temp); // free = 1
    dq->size--; // обращение + декремент = 3
    return value; // return = 1
} // всего: 2 (условие) + 2 + 2 + 4 + 2 (условие) + 3 (худшая из ветвей if)
+ 1 + 3 + 1 = 20

```

```

int pop_back(Deque* dq) {
    if (!dq->first) return -1; // обращение + проверка условия + return = 3

    Node* temp = dq->last; // обращение + присваивание = 2
    int value = temp->edge; // обращение + присваивание = 2
    dq->last = dq->last->prev; // 3 обращения + присваивание = 4

    if (dq->last) { // обращение + проверка условия = 2

```

```

    dq->last->next = NULL; // 2 обращение + присваивание = 3
} else {
    dq->first = NULL; // обращение + присваивание = 2
}

free(temp); // free = 1
dq->size--; // обращение + декремент = 3
return value; // return = 1
} // всего: 2 (условие) + 2 + 2 + 4 + 2 (условие) + 3 (худшая из ветвей if) +
1 + 3 + 1 = 20

void free_deque(Deque* dq) {
    while (dq->first) { // обращение + проверка условия = 2 * n^2 + 2 (где nxn -
размерность матрицы смежности)
        pop_front(dq); // pop_front = 20
    } // общее: (20 + 2) * N^2 + 2 = 22 * n^2 + 2
    free(dq); // free = 1
} // всего: 22 * n^2 + 3

bool is_empty(const Deque* dq) {
    return dq->size == 0; // обращение + сравнение + return = 3
} // всего: 3

int get_element(Deque* dq, int row, int col, int n) {
    Node* current = dq->first; // обращение + присваивание = 2
    int pos = row * n + col; // умножение + сложение + присваивание = 3

    for (int i = 0; i < pos && current; i++) { // присваивание + (сравнение + логическое И
+ инкремент) = 5 (4 на каждой итерации)
        current = current->next; // обращение + присваивание = 2
    } // с учетом общего количества итераций: 1 + (4 + 2) * (row
* n + col) + 2 (проверка условия для конца цикла) = (6 * row) * n + (6 * col + 3)
    return current ? current->edge : 0; // обращение + условие + return = 3
} // всего: 2 + 3 + (6 * row) * n + (6 * col + 3) + 3 = (6 * row)
* n + (6 * col + 11)
// худший случай: 6 * n^2 + 5 (так как row и col меньше n -
> max_row = n - 1 и max_col = n - 1)

int is_valid_number(const char *str) {
    char *endptr;
    errno = 0; // присваивание = 1
    strtod(str, &endptr); // вызов функции + амперсанд (взятие
адреса) = 2

    return (*endptr == '\0' || isspace(*endptr)) && errno != ERANGE; // разыменованье +
сравнение + логическое ИЛИ + разыменованье + isspace + логическое И + сравнение +
return = 8
} // всего: 1 + 2 + 8 = 11

```

```

char* read_dynamic_line(FILE* file) {
    size_t buf_size = INITIAL_BUF_SIZE;           // присваивание = 1
    char* buffer = malloc(buf_size);               // malloc + присваивание = 2
    if (!buffer) return NULL;                       // проверка условия + return = 2

    size_t pos = 0;                                // присваивание = 1
    int c;

    while ((c = fgetc(file)) != EOF && c != '\n') { // fgetc + присваивание + 2 сравнения +
        // логическое И = 5 (на каждой итерации)
        if (pos + 1 >= buf_size) {                  // сложение + сравнение = 2
            buf_size *= 2;                          // умножение + присваивание = 2
            char* new_buf = realloc(buffer, buf_size); // realloc + присваивание = 2
            if (!new_buf) {                          // проверка условия = 1
                free(buffer);                        // free = 1
                return NULL;                        // return = 1
            }
            buffer = new_buf;                        // присваивание = 1
        }
        buffer[pos++] = (char)c;                    // индексация + инкремент + присваивание +
        // приведение типа = 5
    }                                                // общее (для цикла (длина считываемой строки - 1)): 1 *
    // (5 + 2 + 2 + 2 + 1 (условие) + 1 + 5) +
    // 5 (выход из цикла) = 18 * 1 + 5

    if (pos == 0 && c == EOF) {                      // 2 сравнения + логическое И = 3
        free(buffer);                              // free = 1
        return NULL;                              // return = 1
    }

    buffer[pos] = '\0';                            // индексация + присваивание = 2
    return buffer;                                  // return = 1
}                                                  // всего: 1 + 2 + 2 + 1 + 18 * 1 + 5 + 3 (условие) + 2 + 1 =
18 * 1 + 17

int* kahn_topological_sort(Deque* matrix, int n, int* result_size) {
    int* in_degree = calloc(n, sizeof(int));        // sizeof + calloc + присваивание = 3
    if (!in_degree) {                              // проверка условия = 1
        printf("Ошибка...\n");                    // printf = 1
        return NULL;                              // return = 1
    }

    Deque* zero_in_degree = create_deque();         // вызов функции = 11
    if (!zero_in_degree) {                         // проверка условия = 1
        free(in_degree);                          // free = 1
        printf("Ошибка...\n");                    // printf = 1
        return NULL;                              // return = 1
    }

```

```

}

int* result = malloc(n * sizeof(int));    // sizeof + умножение + malloc + присваивание =
4
if (!result) {                            // проверка условия = 1
    free(in_degree);                      // free = 1
    free_deque(zero_in_degree);           // вызов функции = 22 * n^2 + 3
    printf("Ошибка...\n");               // printf = 1
    return NULL;                         // return = 1
}

*result_size = 0;                        // разыменованье + присваивание = 2

for (int i = 0; i < n; i++) {              // инициализация + сравнение + инкремент = 3
    for (int j = 0; j < n; j++) {          // инициализация + сравнение + инкремент = 3
        if (get_element(matrix, i, j, n)) { // вызов функции = 6 * n^2 + 5
            in_degree[j]++;               // индексация + инкремент = 3
        }
    }
}                                          // общее (для циклов): 1 + [1 + (6 * n^2 + 5 + 3) * n + 3n + 1]
* n + 3n + 1 =
                                          // = 2 + 3n + 6 * n^4 + 11 * n^2 + 2n = 6 * n^4 + 11 * n^2 + 5n
+ 2

for (int i = 0; i < n; i++) {              // инициализация + сравнение + инкремент = 3
    if (in_degree[i] == 0) {              // инкремент + сравнение = 2
        push_back(zero_in_degree, i);     // вызов функции = 21
    }
}                                          // общее (для цикла): 1 + (2 + 21) * n + 1 = 23n + 2

while (!is_empty(zero_in_degree)) {        // вызов функции + сравнение = 4
    int v = pop_front(zero_in_degree);     // вызов функции = 20
    result[(*result_size)++] = v;          // разыменованье + инкремент + присваивание = 4

    for (int j = 0; j < n; j++) {          // инициализация + сравнение + инкремент = 3
        if (get_element(matrix, v, j, n)) { // вызов функции = 6 * n^2 + 5
            in_degree[j]--;                // индексация + декремент = 3
            if (in_degree[j] == 0) {        // индексация + сравнение = 2
                push_back(zero_in_degree, j); // вызов функции = 21
            }
        }
    }
}                                          // общее (для цикла): [4 + 20 + 4 + 1 + 1 + (6 * n^2 + 5 + 3 + 2
+ 21) * n] * n + 4 =
                                          // = 4 + [30 + 6 * n^3 + 31n] * n = 6 * n^4 + 31 * n^2 + 30n + 4

free(in_degree);                          // free = 1
free_deque(zero_in_degree);               // вызов функции = 22 * n^2 + 3

```



```

if (*result_size != n) {           // разыменование + сравнение = 2
    free(result);                  // free = 1
    printf("Ошибка...\n");        // printf = 1
    return NULL;                  // return = 1
}

return result;                     // return = 1
}
// всего: 3 + 1 + 11 + 1 + 4 + 1 + 2 + 6 * n^4 + 11 * n^2 + 5n +
2 + 23n + 2 + 6 * n^4 + 31 * n^2 + 30n + 4 + 1 +
// 22 * n^2 + 3 + 2 + 1 + 1 + 1 = 12 * n^4 + 64 * n^2 + 58n + 40
~ n^4

// Вывод: сложность топологической сортировки по схеме
Кана - O(n^4), где n - количество вершин графа

int kmp_search(const char* text, const char* pattern) {
    int n = strlen(text);          // strlen + присваивание = 2
    int m = strlen(pattern);       // strlen + присваивание = 2
    if (m == 0 || n < m) return -1; // 2 сравнения + логическое ИЛИ + return = 4

    int* lps = malloc(m * sizeof(int)); // sizeof + умножение + malloc + присваивание
    = 4
    if (!lps) return -1;           // проверка условия + return = 2

    int len = 0;                   // присваивание = 1
    lps[0] = 0;                    // индексация + присваивание = 2

    for (int i = 1; i < m; ) {      // инициализация + сравнение = 2
        if (pattern[i] == pattern[len]) { // 2 индексации + сравнение = 3
            len++;                  // инкремент = 2
            lps[i] = len;           // индексация + присваивание = 2
            i++;                    // инкремент = 2
        } else {
            if (len != 0) {          // сравнение = 1
                len = lps[len - 1]; // индексация + вычитание + присваивание = 3
            } else {
                lps[i] = 0;          // индексация + присваивание = 1
                i++;                 // инкремент = 2
            }
        }
    }
}
// общее (для цикла): 1 + ((3 + 2 + 2 + 2) + 1) * m = 10m + 1

int i = 0, j = 0;                 // 2 присваивания = 2
while (i < n) {                    // сравнение = 1
    if (pattern[j] == text[i]) {   // 2 индексации + сравнение = 3
        i++;                       // инкремент = 2
        j++;                       // инкремент = 2
    }
}

```

```

    if (j == m) {                                // сравнение = 1
        free(lps);                                // free = 1
        return i - j;                             // вычитание + return = 2
    }
    else if (i < n && pattern[j] != text[i]) {    // 2 индексации + 2 сравнения + логическое И
= 5
        if (j != 0) {                            // сравнение = 1
            j = lps[j - 1];                       // вычитание + индексация + присваивание = 3
        } else {
            i++;                                   // инкремент = 2
        }
    }
}                                                  // общее (для цикла): (3 + 2 + 2 + 1 + 5 + 1 + 3) * n + 1 =
17n + 1

    free(lps);                                    // free = 1
    return -1;                                    // return = 1
}                                                  // всего: 2 + 2 + 3 + 4 + 1 + 1 + 2 + 10m + 1 + 2 + 17n + 1 +
1 + 1 = 17n + 10m + 21 ~ n + m

```

// Вывод: сложность поиска по схеме Кнута-Морриса-Пратта - $O(n + m)$, где n - длина строки из отсортированных вершин графа, разделенных пробелами, а m - длина подстроки, которую нужно найти

```

char* read_input_line() {
    size_t buf_size = INITIAL_BUF_SIZE;          // присваивание = 1
    char* buffer = malloc(buf_size);              // malloc + присваивание = 2
    if (!buffer) return NULL;                     // проверка условия + return = 2

    size_t pos = 0;                               // присваивание = 1
    int c;

    while ((c = getchar()) != EOF && c != '\n') { // getchar + присваивание + 2 сравнения +
логическое И = 5
        if (pos + 1 >= buf_size) {                // сложение + сравнение = 2
            buf_size *= 2;                         // умножение + присваивание = 2
            char* new_buf = realloc(buffer, buf_size); // realloc + присваивание = 2
            if (!new_buf) {                        // проверка условия = 1
                free(buffer);                      // free = 1
                return NULL;                      // return = 1
            }
            buffer = new_buf;                      // присваивание = 1
        }
        buffer[pos++] = (char)c;                  // инкремент + индексация + приведение типа +
присваивание = 4
    }

    if (pos == 0 && c == EOF) {                    // 2 сравнения + логическое И = 3

```

```

    free(buffer);                                // free = 1
    return NULL;                                // return = 1
}

buffer[pos] = '\0';                            // индексация + присваивание = 2
return buffer;                                // return = 1
}                                              // всего (k - длина считываемой строки): 1 + 2 + 2 + 1 + 1
+ ((2 + 2 + 2 + 1 + 1) + 4) * k + 1 + 3 + 2 + 1 = 12k + 14

int main() {
    const char* filename = "matrix.txt";        // присваивание = 1
    FILE* file = fopen(filename, "r");          // fopen + присваивание = 2
    if (!file) {                                // проверка условия = 1
        perror("Ошибка: не удалось открыть файл"); // perror = 1
        return 1;                              // return = 1
    }                                           // общее (для if): 3

    int rows = 0, cols = 0;                    // 2 присваивания = 2
    char* line = NULL;                          // присваивание = 1

    while ((line = read_dynamic_line(file)) != NULL) { // вызов функции (1 -
// количество символов в строке файла) + присваивание + сравнение = 18 * 1 + 19
        if (strlen(line) == 0) {                // strlen + сравнение = 2
            free(line);                          // free = 1
            continue;                            // continue = 1
        }                                       // общее (для if): 4

        int current_cols = 0;                    // присваивание = 1
        char* ptr = line;                       // присваивание = 1
        char* token;

        while (*ptr) {                          // разыменование + сравнение = 2
            while (*ptr && isspace(*ptr)) ptr++; // 2 разыменования + 2
// сравнения + логическое И + isspace + инкремент = 8
            if (!*ptr) break;                    // разыменование + сравнение + break =
3

            token = ptr;                          // присваивание = 1
            while (*ptr && !isspace(*ptr)) ptr++; // 2 разыменования + 2
// сравнения + isspace + логическое И + инкремент = 8

            if (!is_valid_number(token)) {        // вызов функции = 11
                printf("Ошибка: некорректное значение '%s'\n", (int)(ptr - token), token); // printf
= 1
                free(line);                      // free = 1
                fclose(file);                    // fclose = 1
                return 1;                        // return = 1
            }
}

```

```

        current_cols++; // инкремент = 2
    } // общее (для цикла (max_line -
максимальная длина строки)):  $5 + \max\_line * (2 + 8 + 2 + 1 + 45) = 58 * \max\_line + 5$ 

    if (cols == 0) { // сравнение = 1
        cols = current_cols; // присваивание = 1
    } else if (current_cols != cols) { // сравнение = 1
        printf("Ошибка: разное количество элементов в строках\n"); // printf = 1
        free(line); // free = 1
        fclose(file); // fclose = 1
        return 1; // return = 1
    } // общее (для if): 2
    rows++; // инкремент = 2
    free(line); // free = 1
} // общее (для цикла):  $4 + 58 * \max\_line + 5 +$ 
 $2 + 3 = 58 * \max\_line + 14$ 

if (rows == 0 || cols == 0) { // 2 сравнения + логическое ИЛИ = 3
    printf("Ошибка: пустая матрица\n"); // printf = 1
    fclose(file); // fclose = 1
    return 1; // return = 1
} // общее (для if): 6

if (rows != cols) { // сравнение = 1
    printf("Ошибка: матрица смежности должна быть квадратной\n"); // printf = 1
    fclose(file); // fclose = 1
    return 1; // return = 1
} // общее (для if): 4

rewind(file); // rewind = 1

Deque* matrix = create_deque(); // вызов функции = 11
if (!matrix) { // проверка условия = 1
    fclose(file); // fclose = 1
    return 1; // return = 1
} // общее (для if): 3

while ((line = read_dynamic_line(file)) != NULL) { // присваивание + вызов
функции (l - длина строки) + сравнение =  $1 + 18 * 1 + 17 + 1 = 18 * 1 + 19$ 
    if (strlen(line) == 0) { // strlen + сравнение = 2
        free(line); // free = 1
        continue; // continue = 1
    }

    char* ptr = line; // присваивание = 1
    char* token;

    while (*ptr) { // разыменование + сравнение = 2

```

```

while (*ptr && isspace(*ptr)) ptr++; // 2 разыменования + 2
сравнения + isspace + логическое И + инкремент = 8
if (!*ptr) break; // разыменование + сравнение + break =
3

token = ptr; // присваивание = 1
while (*ptr && !isspace(*ptr)) ptr++; // 2 разыменования + 2
сравнения + isspace + логическое И + инкремент = 8

double value = strtod(token, NULL); // strtod + присваивание = 2
if (value != 0) push_back(matrix, 1); // сравнение + вызов функции =
1 + 21 = 22
else push_back(matrix, 0); // вызов функции = 21
}
free(line); // free = 1
} // общее (для цикла (max_line -
максимальная длина строки)): 5 + max_line * (2 + 8 + 2 + 1 + 45) = 58 * max_line + 5
fclose(file); // fclose = 1

printf("Содержимое дека (матрица смежности графа):\n"); // printf = 1
Node* current = matrix->first; // обращение + присваивание = 2
for (int i = 0; i < rows; i++) { // присваивание + сравнение +
инкремент = 4
for (int j = 0; j < cols; j++) { // присваивание + сравнение +
инкремент = 4
if (current) { // проверка условия = 1
printf("%d ", current->edge); // обращение + printf = 2
current = current->next; // обращение + присваивание = 2
}
}
printf("\n"); // printf = 1
} // общее (для циклов): 1 + (4 + (1 + 8n + 1)) *
n + 1 = 8 * n^2 + 6n + 2

int result_size;
int* sorted_vertices = kahn_topological_sort(matrix, rows, &result_size); // вызов функции =
O(n^4)
if (!sorted_vertices) { // проверка условия = 1
free_deque(matrix); // вызов функции = 22 * n^2 + 3
return 1; // return = 1
} // общее (для if): 22 * n^2 + 5

char* sorted_str = malloc(rows * 12); // умножение + malloc +
присваивание = 3
if (!sorted_str) { // проверка условия = 1
printf("Ошибка: не удалось выделить память\n"); // printf = 1
free(sorted_vertices); // free = 1
free_deque(matrix); // вызов функции = 22 * n^2 + 3

```

```

    return 1; // return = 1
} // общее (для if):  $1 + 3 + 22 * n^2 + 3 = 22 * n^2 + 7$ 
sorted_str[0] = '\0'; // индексация + присваивание = 2

for (int i = 0; i < result_size; i++) { // присваивание + сравнение +
инкремент = 4
    char num_str[12];
    sprintf(num_str, "%d ", sorted_vertices[i] + 1); // sprintf + индексация +
    сложение = 3
    strcat(sorted_str, num_str); // strcat = 1
} // общее (для цикла):  $1 + (1 + 3 + 3) * n + 1 = 7n + 2$ 

printf("Отсортированные вершины: %s\n", sorted_str); // printf = 1
printf("Введите подстроку для поиска: "); // printf = 1

char* pattern = read_input_line(); // вызов функции =  $O(n)$ 
if (!pattern) { // проверка условия = 1
    printf("Ошибка: не удалось прочитать ввод\n"); // printf = 1
    free(sorted_vertices); // free = 1
    free(sorted_str); // free = 1
    free_deque(matrix); // вызов функции =  $22 * n^2 + 3$ 
    return 1; // return = 1
} // общее (для if):  $1 + 3 + 1 + 22 * n^2 + 3 = 22 * n^2 + 8$ 

int pos = kmp_search(sorted_str, pattern); // вызов функции =  $O(n + m)$ 
if (pos != -1) { // сравнение = 1
    printf("Подстрока '%s' найдена на позиции: %d\n", pattern, pos + 1); // printf = 1
} else {
    printf("Подстрока '%s' не найдена\n", pattern); // printf = 1
} // общее (для if):  $1 + 1 = 2$ 

free(sorted_vertices); // free = 1
free(sorted_str); // free = 1
free(pattern); // free = 1
free_deque(matrix); // вызов функции =  $22 * n^2 + 3$ 

return 0; // return = 1
}

// Общая сложность: если просмотреть все
общие сложности сложных кусков кода, то
будет  $O(n^4)$  // можно заметить, что сложность программы

```

Основные блоки:

- Получение элемента: $O(n^2)$
- Топологическая сортировка по схеме Кана: $O(n^4)$
- Поиск по схеме Кнута-Морриса-Пратта: $O(n + m)$
- Вся программа: $O(n^4)$

Сложность топологической сортировки по схеме Кана обычно оценивается следующим образом: $O(V + E)$, где V – число вершин, E – число ребер. Я рассматривала критические случаи (то есть пусть число вершин равно n , а число ребер – n^2). Тогда можно было бы сказать, что сложность алгоритма $O(n^2)$, но в самой сортировке используется еще и функция получения элемента из матрицы, которая хранится в виде дека на базе связного списка, что усложняет доступ к ее элементам. Поэтому возрастает сложность алгоритма.

Оценка времени работы программы

